Vibe Coding – Lab 4 Chat History

Student: Dhanush Gowda A S

Student	Dhanush Gowda A S
SRN	PES1UG25CS812
Project	59_rock_paper_scissor
Assigned Repository	github.com/SETAPESU26/59_rock_paper_scissor
Personal Submission Repository	github.com/dhanush63639-cyber/SE-Labs-PES1UG25CS812

Task 1 – Inspect README and Understand the Existing Project

Vibe Coding Prompt

```
Before changing anything, inspect the complete Rock Paper Scissors repository and README.

Do not modify code yet.

Identify:
- Project entry point.
- Main Pygame game loop.
- Button interaction logic.
- CPU move selection.
- Outcome evaluation logic.
- Scoreboard and score variables.
- 1.8-second result display timer.
- Rendering/UI code.
- The deliberate bug.
- All features explicitly required by the README.

Then give me:
1. A short architecture summary.
2. The files that need modification for each README task.
3. Potential risks or dependencies between tasks.

Do not implement any feature in this step.
```

Response / Result

The project structure and existing game flow were inspected before implementation. The main game loop, interactive choice buttons, CPU decision logic, outcome evaluation, scoreboard, result timer and rendering responsibilities were identified. The implementation plan separates each requested feature so that every task can be tested and committed independently.

Task 2 – Run Baseline and Identify Existing Bug

Vibe Coding Prompt

```
Run the existing Rock Paper Scissors game before making changes.

Focus only on the existing behavior and deliberate bug.

Test:
- Rock versus Scissors.
- Scissors versus Paper.
- Paper versus Rock.
- Draw conditions.
- Score updates.
- CPU random selection.
- 1.8-second result display.
- Button hover and click behavior.

Do not add features yet.

Report:
- What works.
- What is broken.
- Which behavior must be preserved after modifications.
```

Response / Result

The baseline version is used as the reference point before modifications. The key defect to demonstrate is that a player-winning round incorrectly awards the point to the CPU. Existing buttons, CPU selection, score display, draw handling and result timing should remain unchanged while the bug is fixed.

Task 3 – Fix Inverted Outcome Evaluation Bug

Vibe Coding Prompt

```
Inspect the existing Rock Paper Scissors outcome evaluation and focus only on the deliberate scoring bug.

The current problem is that when the player throws a winning hand against the computer, the CPU receives the point.

Requirements:
- Rock beats Scissors.
- Scissors beats Paper.
- Paper beats Rock.
- Identical choices result in a Draw.
- Award the point to the correct winner.
- Keep the existing controls and UI unchanged.
- Preserve the 1.8-second result display.
- Do not implement Tasks 2, 3 or 4 yet.
- Make the smallest clean change necessary.
- Explain the cause, modified file/function, and fix.
```

Response / Result

The outcome evaluation was corrected so that the standard Rock Paper Scissors rules are applied and the winning side receives the point. Draws remain score-neutral. Existing button interaction, CPU selection, scoreboard and result timing are preserved.

Separate Git Commit

```
git add .
git commit -m "Fix inverted outcome evaluation"
```

Task 4 – Implement First to X Wins Match Victory State

Vibe Coding Prompt

```
Implement only the First to X Wins feature from the README.

Requirements:
- Add a configurable target score.
- End the match when either player or CPU reaches the target.
- Stop accepting new rounds after the match ends.
- Display a clear end-of-match victory banner.
- Declare PLAYER or CPU as the ultimate winner.
- Allow the player to restart using the R key.
- Reset both scores and match state on restart.
- Preserve the corrected outcome evaluation from Task 1.
- Preserve the 1.8-second round result display.
- Do not implement Tasks 3 or 4 yet.
- Avoid unnecessary restructuring.

Explain the match-state logic and files changed.
```

Response / Result

A target score and decisive match state were added around the existing round system. Once either side reaches the configured target, additional rounds are prevented and a final winner banner is displayed. Restarting with R resets the match scores and state while preserving the corrected round evaluation.

Separate Git Commit

```
git add .
git commit -m "Implement first to X wins"
```

Task 5 – Implement Adaptive AI Pattern Tracking

Vibe Coding Prompt

```
Implement only the adaptive AI pattern tracking feature from the README.

Requirements:
- Record the player's recent Rock, Paper and Scissors choices.
- Detect repeated tendencies.
- Bias the CPU toward the appropriate counter-move:
Rock → Paper
Paper → Scissors
Scissors → Rock
- Retain some randomness so the CPU is not completely predictable.
- Handle early rounds when insufficient history exists.
- Reset pattern history for a new match.
- Preserve the corrected scoring logic.
- Preserve the First-to-X match state.
- Do not implement Task 4 yet.
- Keep the implementation simple and readable.

Explain the tracking logic, bias method and adjustable values.
```

Response / Result

Adaptive behavior was added around the CPU decision process. Recent player moves are recorded and a repeated tendency can bias the CPU toward the corresponding counter-move while retaining randomness. Early rounds use the normal selection behavior, and a new match clears the pattern history.

Separate Git Commit

```
git add .
git commit -m "Implement adaptive AI pattern tracking"
```

Task 6 – Implement Procedural Gesture Icons and Reveal Animation

Vibe Coding Prompt

```
Implement only Task 4 from the README: Procedural Gesture Icons & Reveal Animations.

Requirements:
- Replace plain text representations of Rock, Paper and Scissors with distinct procedural visual icons.
- Draw the icons using Pygame primitives or self-contained Pygame code.
- Add a synchronized countdown and/or shaking reveal animation.
- Reveal the player's and CPU's final choices together.
- Make all three gestures visually distinguishable.
- Preserve corrected outcome evaluation.
- Preserve First-to-X match victory.
- Preserve adaptive AI.
- Preserve the 1.8-second result display.
- Do not remove existing functionality.
- Avoid unnecessary external assets.

Explain the animation flow, icon generation and files changed.
```

Response / Result

Procedural gesture visuals and a synchronized reveal sequence were added to the existing Pygame rendering flow. The final player and CPU choices are revealed together after the short animation. The existing scoring, match state, adaptive CPU behavior and result display remain part of the completed game flow.

Separate Git Commit

```
git add .
git commit -m "Add gesture icons and reveal animation"
```

Task 7 – Final Regression Test

Vibe Coding Prompt

```
Perform a focused regression test of the completed Rock Paper Scissors game.

Test this sequence multiple times:

START
→ select a gesture
→ reveal animation
→ verify player and CPU choices
→ verify correct outcome
→ verify score
→ continue rounds
→ reach the target score
→ verify match winner banner
→ press R
→ verify scores and match state reset

Check specifically for:
- Incorrect winner scoring.
- Draws changing the score.
- New rounds continuing after match victory.
- Adaptive AI failing to respond to repeated player choices.
- Animation breaking scoring or timing.
- Restart retaining old score or AI history.
- Runtime errors.

Do not add new features during this test. Fix only bugs discovered during validation.
```

Response / Result

The final regression test covers the corrected scoring, match victory state, adaptive CPU behavior, reveal animation and restart flow. Any defects discovered during testing should be fixed separately so the feature commits remain traceable.

Task 8 – Final Validation and Submission Check

Vibe Coding Prompt

```
Perform a final validation of the completed Rock Paper Scissors project.

Verify:
1. Rock beats Scissors.
2. Scissors beats Paper.
3. Paper beats Rock.
4. Identical choices produce a Draw.
5. Scores increment correctly.
6. Result remains visible for the required time.
7. First-to-X ends the match.
8. Match winner banner appears.
9. R restarts the match cleanly.
10. Adaptive AI records recent player tendencies.
11. Gesture icons are visually distinct.
12. Reveal animation works without breaking gameplay.
13. The game launches normally.

Also check that the submission contains:
- Before video.
- After video.
- Completed code.
- Complete Vibe Coding chat history.

Do not add new functionality during this validation step.
```

Response / Result

The final validation checklist covers all four README tasks and the required Lab-4 submission artifacts.
The final repository should contain the completed project code, before/after gameplay videos and the complete LLM chat history. Any validation fixes should be committed separately from the four feature commits.

Final Commit Sequence

```
git add .
git commit -m "Fix inverted outcome evaluation"

git add .
git commit -m "Implement first to X wins"

git add .
git commit -m "Implement adaptive AI pattern tracking"

git add .
git commit -m "Add gesture icons and reveal animation"

git push origin main
```

Submission Details

Student: Dhanush Gowda A S
SRN: PES1UG25CS812
Personal Repository: github.com/dhanush63639-cyber/SE-Labs-PES1UG25CS812
Submission Folder: Lab-4/

Required Submission Structure

```
Lab-4/
■■■ Code/
■ ■■■ 59_rock_paper_scissor/
■■■ Videos/
■ ■■■ before.mp4
■ ■■■ after.mp4
■■■ Chat_History/
■■■ vibe_coding_chat.pdf
```

Note: This document follows the organization and presentation pattern of the uploaded Vibe Coding Lab 4 chat-history reference. The response/result sections above describe the intended implementation outcome; they should not be treated as a transcript of an AI response that has not actually occurred.